

Tutorial Git

Antes de comenzar con Git, primero daremos unas pequeñas notas sobre él:

- Git es un sistema distribuido de control de versiones que no depende de un repositorio central.
- Los clientes replican completamente el repositorio, no parte de él.
- Git es un proyecto de software libre que fue lanzado por Linus Torvars.
- Git ofrece un historial de revisiones completo.
- Se puede trabajar en varias ramas y realizar fusiones de forma ágil.

Terminología de los sistemas de gestión de versiones

- **Repositorio:** lugar donde se almacenan todos los datos y cambios. Puede ser un sistema de archivos en un disco duro, un banco de datos, un servidor, etc.
- **Versión:** estado concreto de los datos almacenados. Algunos sistemas identifican las versiones con un número contador. Otros identifican las versiones mediante un código de detección de modificaciones. La última versión se identifica como la cabeza o **HEAD**.
- **HEAD:** hace referencia a la versión más reciente de una determinada rama o del tronco. El tronco y cada rama tienen su propio padre, y para referirse al padre se suele utilizar el término HEAD.
- **Tronco (trunk).** Es la línea principal de desarrollo de un proyecto.
- **Rama (branch).** Son copias de archivos, carpetas o proyectos. cuando se crea una rama se crea una bifurcación del proyecto y se crean dos líneas de desarrollo. Son motivos habituales de creación de ramas la creación de nuevas funcionalidades o corrección de errores.
- **Desplegar (checkout).** Crear una copia de trabajo del proyecto, de la rama o de archivos y carpetas del repositorio en el equipo local. Se puede seleccionar una versión concreta.
- **Confirmar (commit).** Confirmar los cambios realizados en local para integrarlos en el repositorio.
- **Pull.** copiar los cambios de un repositorio (remoto, por lo general) en el depósito local.
- **Push:** Acciones utilizadas para añadir los cambios del repositorio local a otro repositorio que habitualmente está en remoto.

Las áreas de trabajo de Git

Anteriormente hemos visto los estados que puede tener un fichero en Git, lo cual está muy relacionado con las áreas de trabajo. Estas áreas son:

- **El directorio de trabajo (o `working`)**. Es el directorio local de trabajo, donde creamos los archivos, y realizamos todas las modificaciones. Cuando se realiza una modificación de un archivo pasa al área **`working`**.
- **El área de `staging`**. Es un área intermedia en el que se prepara a los archivos para luego validarlos en el repositorio local. Los cambios y modificaciones de los archivos del área de **`working`**, antes de validarse y pasar al área de **`commit`**, pasarán por el área de **`staging`**.
- El **repositorio local** o área de `commit`. Lo creamos con GIT, y es donde se van a guardar nuestros archivos con las modificaciones y todas las versiones por las que ha pasado. Normalmente pasaremos los archivos del área de preparación, al área de **`commit`**.
- El **repositorio remoto**. Estará fuera del equipo local, que, en caso de colaborar con más usuarios, podremos sincronizar nuestros ficheros y cambios realizados con el repositorio remoto (github, gitlab, bitbucket, etc. son herramientas que nos permiten crear repositorios remotos).

Estados de un fichero Git

Los tres estados por los que pueden pasar los ficheros son los siguientes:

- **Modificado (`untracked`)**: el archivo ha sido modificado, pero no ha sido confirmado a tu base de datos.
- **Preparado (`staged`)**: el archivo ha sido marcado como modificado para que vaya a su próxima confirmación.
- **Confirmado (`committed`)**: los datos del archivo están almacenados de forma segura en la base de datos local. Es un snapshot al que podremos volver en un momento determinado.

Operaciones básicas de git

git init: inicializas un directorio de trabajo como un repositorio local de Git.

git status: Muestra el estado de un repositorio local de git.

- En **rojo** aparecerán aquellos ficheros que se encuentran *modificados*.
- En **verde** aparecerán aquellos ficheros que se encuentran *preparados*.
- Los ficheros *confirmados* no aparecerán.

Escenarios

- No hay nada modificado
- Hay algún fichero modificado (App.java)

```
→ ejercicioGit git:(master) ✕ git status
En la rama master
Cambios no rastreados para el commit:
  (usa "git add <archivo>..." para actualizar lo que será confirmado)
  (usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
      modificado:    README.md

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
```

- Ficheros en staging area

```
[→ ejercicioGit git:(master) ✕ git status
En la rama master
Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
      modificado:    README.md
```

- Ficheros confirmados

```
[→ ejercicioGit git:(master) git status
En la rama master
nada para hacer commit, el árbol de trabajo está limpio
→ ejercicioGit git:(master) █
```

git config --global user.name: establece el nombre del usuario que va a figurar como autor del commit, que podremos ver en los logs.

git config --global user.email: establece el email del usuario que va a figurar como autor del commit, que podremos ver en los logs.

`git add <fichero(s)>`: permite *preparar* un fichero que se encontraba *modificado*. En el caso de querer *confirmar* todos los ficheros a la vez, se puede utilizar el mandato `git add .`

`git commit -m "<mensaje>"`: permite *confirmar* los ficheros que aparecen como *preparados*. La opción `-m` nos permite añadir un mensaje al commit.

`git log --oneline`: muestra todos los commits que tiene el proyecto en una única línea.

`git remote add origin <url>`: asociamos un repositorio remoto (url) a nuestro repositorio local.

`git pull origin master`: descargamos el repositorio remoto a nuestro repositorio local. Concretamente, descargaremos la rama llamada `master`.

`git push origin master`: subimos nuestro repositorio local al repositorio remoto. Concretamente a la rama llamada `master`.

`git checkout <commit>`: reemplazará todo lo que tengamos en el área de working por el commit que indiquemos.

`git checkout <nombreRama>`: se utiliza para realizar el checkout de una rama.

Escenarios de aplicación

A continuación mostraré diferentes escenarios que hacen uso de los mandatos anteriores.

Inicializar un repositorio local y hacer el primer commit

En este escenario veremos como inicializar un repositorio local y realizar un commit con su contenido.

1. Nos moveremos a la carpeta que será el repositorio local.
2. Inicializamos el repositorio local de git.

```
git init
```

3. Cambiamos el estado de los ficheros *modificados* a *preparados*.

```
git add .
```

4. Cambiamos el estado de los ficheros *preparados* a *confirmados*.

```
git commit -m "First Commit"
```

Confirmar modificaciones en repositorio

En este escenario veremos como es posible confirmar los cambios realizados en un fichero.

1. Crearemos un nuevo fichero

```
echo "# Primer proyecto GIT" > readme.md
```

2. Veremos el estado del proyecto y confirmaremos que el fichero readme.md se encuentra en working directory.

```
git status
```

3. Modificamos el estado del fichero readme.md a *preparado*.

```
git add readme.md
```

4. Confirmamos los cambios

```
git commit -m "añado fichero readme.md"
```

Ver información del repositorio

En este escenario mostraré varias formas de ver información sobre el repositorio.

- git log: mostrará todas las revisiones del repositorio con información como el autor, la fecha, el mensaje, el identificador del commit, ...

- `git log --oneline`: permite mostrar información de los commits de una forma más abreviada y mostrando la información más relevante
- `git log --pretty="%h %an %ar - %s"`: con esta instrucción hemos configurado la información y el formato en el que se debe mostrar.
- `git show`: permite mostrar los cambios realizados en todos los commits.
- `git show idCommit`: muestra los cambios realizados en un commit en concreto.

Deshacer cambios

Una de las propiedades de los sistemas de control de versiones es la propiedad de deshacer cambios y volver a un momento anterior. Es importante tener en cuenta los cambios que se hacen en el área de working, si se borran de working no se pueden recuperar, se recuperarán si antes han pasado por las áreas de staging o commit.

Deshacer un cambio en working desde el último commit.

1. Modifico el fichero `readme.md`

```
echo "## otra linea" >> readme.md
```

2. Compruebo que está en el working directory, pero no en el staging área.

```
git status
```

3. Elimino la modificación reemplazando todo el working directory por su estado en commit.

```
git checkout .
```

Deshacer un cambio en working desde staging área.

1. Modifico el fichero `readme.md`

```
echo "## nueva linea" >> readme.md
```

2. Añado dicha modificación a staging área

```
git add readme.md
```

3. Devolver el estado del fichero en staging a working

```
git reset HEAD readme.md
```

CUIDADO

`git reset --hard` combina tanto el efecto de `git checkout` como el de `git reset` en un único comando. Se eliminan los ficheros del área staging y del área de working de tal forma que volvemos a los mismos contenidos que estaban presentes en el último commit.

```
git reset --hard HEAD
```

Deshacer un cambio creando un nuevo commit

1. Modifico el fichero readme.md

```
echo "## nueva linea" >> readme.md
```

2. Añado dicha modificación a staging área

```
git add readme.md
```

3. Confirmo los cambios

```
git commit -m "nuevo cambio antes de revert"
```

4. Revierto los cambios de un commit antes del HEAD

```
git revert HEAD
```

Se pueden revertir los commits hechos entre HEAD y HEAD~2 de la siguiente forma:

```
git revert HEAD...HEAD~2
```

Convertir un proyecto local en un repositorio y subirlo a GitHub

Partiremos de la base de que tenemos un proyecto local en Java que ha sido creado con eclipse, dicho proyecto se encuentra en la carpeta *MongoDBProject*. Queremos compartir dicho proyecto con la comunidad de desarrolladores y para ellos utilizaremos la plataforma de GitHub.

1. El primer paso será crear un repositorio en GitHub que contenga el fichero README.md que se crea automáticamente.
2. El segundo paso será copiar la URL de dicho proyecto, en nuestro caso será <https://github.com/.....>
3. Nos moveremos al directorio local de nuestro proyecto (cualquiera implementado con un IDE) y lo iniciaremos como repositorio local.

```
git init
```

4. Asociaremos un repositorio remoto al repositorio local que acabamos de crear.

```
git remote add origin <url>
```

5. Traer los ficheros que hay en el repositorio remoto (sólo está el README.md)

```
git pull origin master
```

```
git pull --allow-unrelated-histories origin master
```

6. Pasaremos los cambios a *staged area*.

```
git add .
```

7. Confirmamos los cambios.

```
git commit -m "mensaje del commit"
```

8. Subir los cambios al servidor

```
git push origin master
```

Traer a local un repositorio remoto (clonar)

1. Crear la carpeta donde queremos clonar el repositorio, es decir, la carpeta que queremos que contenga el código fuente de nuestro proyecto remoto.
2. Una vez situado dentro de la carpeta, ejecuta el siguiente comando para clonar el repositorio remoto:

```
git clone <url>
```

Nota: internamente este comando ejecutará las instrucciones:

- (1) `git init` para inicializar el repositorio local;
- (2) `git remote add origin <url>` para asignarle un repositorio remoto; y
- (3) `git pull origin master` para traer el contenido del repositorio remoto.

Trabajar con ramas

Para la creación de ramas utilizaremos el comando

```
git branch <nombreDeLaRama>
```

Se podrán ver las ramas que tenemos con

```
git branch -v -a
```

Para cambiar de rama ejecutaremos

```
git checkout <nombreRama>
```

Crear una rama y moverme a ella

1. Lo primero que haremos será ver el estado del proyecto y sus ramas.

```
git branch -v -a
```

2. Crearemos una nueva rama llamada “prueba”.

```
git branch prueba
```

3. Volvemos a ver el estado del proyecto y sus ramas. Veremos que la nueva rama se ha creado.

```
git branch -v -a
```

4. Aunque la rama se haya creado, no hemos cambiado de rama. Para cambiar de rama ejecutaremos:

```
git checkout prueba
```

Fusionar ramas

Partiendo del anterior escenario, haremos ciertas modificaciones en la rama prueba y las añadiremos a la rama master.

1. Nos aseguramos que estamos en la rama pruebas

```
git checkout pruebas
```

2. Realizamos modificaciones en la rama

```
echo "fich1" >> fichero1.txt
```

```
echo "fich2" >> fichero2.txt
```

```
echo "fich2" >> fichero3.txt
```

3. Añadimos estos nuevos cambios a staging área y realizamos el commit.

```
git add .
```

```
git commit -m "nuevos ficheros"
```

4. Comprobamos el estado del repositorio con git log. Veremos que HEAD apunta a prueba y master se encuentra en otro commit.

```
git log --oneline
```

También podremos ejecutar

```
git branch -v -a
```

5. Cambiaremos de rama a master otra vez

```
git checkout master
```

6. Accederemos al contenido del repositorio y veremos que los ficheros creados anteriormente no existen en esta rama.

```
ls -l
```

7. Ahora fusionaremos las ramas. Queremos fusionar master con pruebas, por lo que situándonos en la rama master, ejecutaremos el siguiente mandato:

```
git merge prueba
```

8. Comprobaremos el estado del proyecto

Subir la rama a remoto y eliminar la rama de remoto

Cuando queremos subir el contenido de una rama a un repositorio remoto, escribiremos lo siguiente:

```
git push origin master <nombreRama>
```

En nuestro caso sería:

```
git push origin master prueba
```

Si vamos a GitHub veremos que existe la nueva rama. Si queremos eliminar dicha rama, ejecutaremos el mandato:

```
git push origin --delete prueba
```

Si queremos eliminar una rama en local ejecutaremos:

```
git branch -d prueba
```

Traer una rama remota

Si quieres descargar una rama del repositorio remoto al local ejecutaríamos:

```
git fetch origin prueba
```

Para situarnos en la nueva rama ejecutamos:

```
git checkout prueba
```

Ejercicio

Crea en el disco la carpeta **ejercicioGit**, e inicia Git Bash.

1. Convierte la carpeta en un repositorio local
2. Crea dos archivos (arch1.txt y arch2.txt), añade texto, y confírmalos.
3. Realiza cambios en arch1.txt y confirma dichos cambios.
4. Realiza cambios en arch2.txt y confirma los cambios.
5. Utiliza revert para volver a la situación del Proyecto antes de haber realizado los cambios en arch1.txt y arch2.txt. Es decir, volver a la confirmación del punto 2.
6. Crea el fichero doc3.txt, añade texto, y confirma los cambios.
7. Crea el repositorio público en GitHub que se llame **ejercicioGit-ed**. Inicialízalo con el fichero README.md

(FOTO)

8. Conectate al repositorio remote y sube lo que tengas en el repositorio local.
9. Crea una nueva rama llamada **rama1**
10. Muevete a **rama1** y crea dos ficheros rama1.txt y rama2.txt, añade varias líneas en ellos, y validalos.
11. Sube **rama1** al repositorio remoto.
12. Situate en la rama master local y fusionala con la **rama1**.
13. Sincroniza la rama master local con la remota.
14. Elimina **rama1** del repositorio local
15. Elimina **rama1** del repositorio remote